4. Neural Networks: The Nine-Step Workflow

Your guide to the most comprehensive Neural Network workflow

Previous Section:

3. Tensors in Deep Learning

Contents:

Deep Learning often looks complicated at first glance. There are models, layers, optimizers, losses, metrics, training loops, and it can feel like a huge system with too many moving parts.

But if we step back, something interesting appears: At its core, deep learning is just a **structured workflow for learning from data**.

Instead of thinking of it as isolated concepts, we treat it as a **continuous pipeline**—a sequence of steps where raw data slowly transforms into predictions, decisions, and intelligence.

This section breaks everything down into **nine core steps** used in almost every deep learning project. We will implement them using **TensorFlow and Keras**, two of the most widely used deep learning frameworks.

Before we begin, make sure your environment is ready:

- On Mac:

```
!pip install tensorflow keras
```

- On Windows:

```
%pip install tensorflow keras
```

These libraries will allow us to build, train, and evaluate neural networks with minimal complexity while keeping full control over the learning process.

The Nine-Step Workflow

Every deep learning system—no matter how complex—follows a similar structure. Once you understand this pipeline, you can apply it to almost any problem: classification, regression, image recognition, or even language models.

Here are the nine essential steps:

1. Load the Data

Everything begins with data. We import datasets from files, databases, or APIs. This is the raw material the model will learn from. At this stage, the data is usually unprocessed—it may

2. Prepare the Data

Raw data cannot be directly used by a neural network. So we transform it into a usable format:

- Normalize numerical values

3. Build the Model (Layers + Activation Functions)

Now we design the architecture of the neural network.

This is where we define:

- How many layers the model has

contain noise, missing values, or unstructured formats. But that is perfectly fine. Deep learning starts from raw reality, not cleaned perfection.

- Encode categorical variables
- Handle missing data
- Split into training, validation, and test sets

This step is crucial because the model is extremely sensitive to how data is represented. A small mistake here can completely change learning behavior later.

- How many neurons per layer
- Which activation functions to use (ReLU, Sigmoid, Softmax, etc.)

Think of this step as designing the **thinking structure** of the model.

If data is experience, then the model architecture is the **brain that processes it**.

4. Compile the Model (Optimizer, Loss, Metrics)

Before training, we must define how the model learns.

This involves three key components:

- **Loss function** → measures how wrong the model is
- **Optimizer** → updates weights to reduce error
- **Metrics** → tracks performance (accuracy, precision, etc.)

At this stage, we are not learning yet—we are defining the **rules of learning**.

5. Train and Validate the Model

Now the learning begins.

We feed the training data into the model, and it starts adjusting its internal weights through backpropagation.

At the same time, we use validation data to check whether the model is actually learning patterns—or just memorizing.

This step is where intelligence slowly emerges from repetition.

6. Fit the Model and Monitor Performance

Training is not just running once—it is an iterative process.

We observe:

- Training loss
- Validation loss
- Training accuracy
- Validation accuracy

These values help us understand whether the model is improving, stagnating, or overfitting.

Often, we visualize this using **loss and accuracy curves**, because patterns in these graphs tell us more than raw numbers.

7. Select the Best Model (Epoch Selection)

Not every training epoch produces the best model.

Sometimes performance improves and then starts

8. Evaluate on the Test Set

Now we test the model on completely unseen data.

This is the real exam.

Unlike training and validation, the test set has

9. Make Predictions

Finally, the model is used in the real world.

We give it new inputs, and it produces outputs:

- Class labels

degrading due to overfitting.

So we select:

- The epoch where validation performance is highest
- Or the model state that generalizes best

This step is about **choosing intelligence, not just training it**.

never influenced the model before. It tells us:

"Did the model truly learn patterns, or just memorize training data?"

This is the final measure of generalization.

- Probabilities
- Numerical predictions

At this stage, the model is no longer learning—it is applying what it has learned.

This is where deep learning becomes useful: when it stops being theory and starts becoming decision-making.

Connecting the Whole Idea

If we zoom out, the entire workflow becomes one simple loop:

$$\text{Data} \rightarrow \text{Processing} \rightarrow \text{Model} \rightarrow \text{Learning} \rightarrow \text{Evaluation} \rightarrow \text{Improvement} \rightarrow \text{Prediction}$$

Each step is necessary. Remove one, and the system breaks.

Deep learning is not magic. It is a **carefully structured process that turns raw data into learned behavior through repetition and optimization**.

Step 1. Load the Data

Now that we understand the full pipeline, we begin where everything starts: **Data.** Because no matter how advanced your neural network is, without data, it does absolutely nothing.

Data comes from everywhere. Sometimes it's clean and curated, sometimes it's messy and chaotic—but it always exists **outside the model**. You might get it from:

- Kaggle → datasets shared by the community
- UCI Machine Learning Repository → classic academic datasets
- GitHub → real-world, project-based data
- Or even your own collected data

And thankfully, we don't always have to start from scratch. There are libraries that already provide datasets ready to use.

In **Scikit-learn**, you'll often see small, structured datasets like:

- Iris dataset
- Breast cancer dataset
- Housing dataset
- Wine dataset

These are usually simple, clean, and perfect for learning. Tables. Rows. Columns. Everything already organized.

But when we move into **deep learning**, things change. Frameworks like TensorFlow provide datasets such as:

- MNIST → handwritten digits
- Boston Housing → regression problem
- IMDB → sentiment analysis (text)
- Reuters → news classification

And here's the important shift: These datasets are often **unstructured**. Not neat tables, but images, text, sequences.

So when we say "load the data", we're not just reading a file. We are **pulling raw reality into the system**. And at this stage, the model still understands **nothing**. No meaning. No labels. No patterns. Just data.

Step 2. Prepare the Data

Here comes the part everyone underestimates. And then regrets. **Data Preparation.**

This is usually the most annoying part. But more importantly, i**t is the most important part.** Because a neural network doesn't fail because it is "not powerful enough". It fails because the data was not prepared in a way it could understand.

Think about it like this. A neural network does not see images, words, or ategories. It only sees ***numbers***. So everything, no matter what it is, must be converted into a numerical form.

That means we often have to:

- **Encode** categorical data (turn labels into numbers)
- **Scale** numerical values (so one feature doesn't dominate others)
- **Vectorize** text (turn words into numerical representations)
- **Normalize** inputs (especially for neural networks)

And here's the real difficulty: You cannot prepare data if you don't understand what it is.

Let's take a very famous example: **MNIST dataset.** At first glance, it sounds simple: "Handwritten digits from students". So naturally, you think: "Oh, it's just numbers from 0 to 9". But when we actually load it, something strange happens.

You print the data, and instead of seeing digits. You see this: A grid. A matrix. Just numbers. That's because each image is stored as a **2D array of pixel values**.

- Each value represents intensity: 0 → black; Higher values → brighter pixels

Since MNIST is **grayscale**, it doesn't use colors like Red, Green, Blue. It only represents how dark, or how light each pixel is. So instead of "This is a 7". The computer sees a matrix of numbers.

And if this surprises you, it shouldn't. Because this is exactly how machines see the world.

Now here comes the interesting question: If the input is just a matrix, then where is the label? Because a neural network needs two things: Input (the image) → Output (the correct answer)

And the answer is: The label is stored separately. So the dataset actually looks like this: X → images (matrices); y → labels (0–9). Without the labels, the model has no idea what it is looking at.

It would just see patterns with no meaning. And this is where beginners often get confused. They think: ***"The image already shows a number... why does the model not understand?"*** → Because the model does not "see" the image like you do. It does not recognize shapes. It does not understand digits. It only sees numbers arranged in space.

So Data Preparation is really about one thing: Bridging the gap between **human understanding** and **machine representation**. And once that bridge is built, only then can the model start learning.

In the upcoming step, we will finally build the model itself. And for the first time, you'll see how a neural network takes these meaningless numbers and slowly turns them into understanding.

Loading the MNIST Dataset (First Contact with Data)

Before training, this is the moment where things become real. Up until now, we've been talking about data in theory. Now we actually **touch it**.

```
# Load the MNIST Dataset
from keras.datasets import mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_d
ata()

print("Train Images length:", len(train_images))
print("Train Labels length:", len(train_labels))
print("Test Images length:", len(test_images))
print("Test Labels length:", len(test_labels))

print("\nTrain Images shape:", train_images.shape)
print("Train Labels shape:", train_labels.shape)
print("Test Images shape:", test_images.shape)
print("Test Labels shape:", test_labels.shape)
```

```
Train Images length: 60000
Train Labels length: 60000
Test Images length: 10000
Test Labels length: 10000

Train Images shape: (60000, 28, 28)
Train Labels shape: (60000,)
Test Images shape: (10000, 28, 28)
Test Labels shape: (10000,)
```

The Length - 60,000 training images; 60,000 training labels; 10,000 testing images; 10,000 testing labels. This is good. Balanced. Clean. Every image has a corresponding label. So structurally everything is correct.

The Shape (This Is Where It Gets Interesting) - Let's focus on this line:

```
# Output: Train Images shape: (60000, 28, 28)
```

This tells us something very precise: There are **60,000 images;** Each image is **28 × 28 pixels.** So instead of thinking: ***"I have 60,000 pictures"*** → The machine sees: ***"I have 60,000 matrices... each of size 28 by 28"*** → Each image is not an image. It is a grid of numbers.

Now look at the labels:

```
# Output: Train Labels shape: (60000,)
```

This means: 60,000 values; Each value is a single number (0–9)

So the dataset is actually:

- `train_images[i]` → a 28×28 matrix
- `train_labels[i]` → the correct digit for that matrix

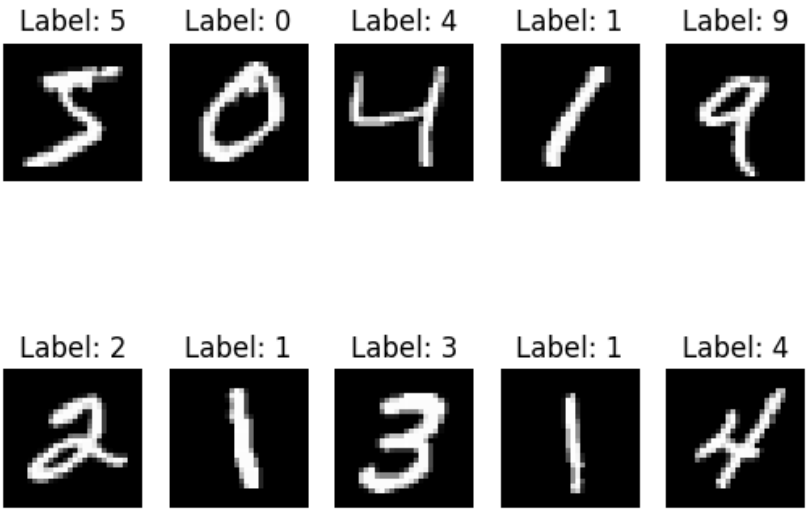
Let's Peek at the Labels

```
print(train_labels[:10])
```

[5 0 4 1 9 2 1 3 1 4]

Show the first ten images and their labels:

```
import matplotlib.pyplot as plt

# Display images with matplotlib
for i in range(1, 2):
    for j in range(10):
        plt.subplot(i + 1, 5, j + 1)
        plt.imshow(train_images[j], cmap='gray')
        plt.title(f"Label: {train_labels[j]}")
        plt.axis('off')

plt.show()
```



Simple. Clean. Almost too clean. At this point, everything "*looks*" ready. We have: Inputs (images); Outputs (labels); A large dataset. So naturally, you might think: "We can train the model now." → ***Not quite.***

Look at the Shape Again: `(60000, 28, 28)`

This is the key issue. Because a neural network—especially a basic one—does not understand: 2D structure. It does not understand: Rows; Columns; Spatial relationships. To the network, this is just **a nested structure of numbers**.

And here's the deeper problem. The model expects input in a very specific format. Usually something like: `(number_of_samples, number_of_features)`

But right now we have: `(number_of_samples, height, width)` → That extra structure is a problem.

Right now, each image is: $28 \times 28 = 784$ values → But the model doesn't see "784 features". It sees: A 28-by-28 grid without knowing what to do with it. So even though the data is **technically correct.** It is not yet in a form the model can learn from.

This is the moment where most people understand something important: Loading data is easy. Understanding data is not.

We are not just preparing data for the model. We are translating it. From: Human-friendly representation (images) → Into: Machine-friendly structure (numerical features)

Because until we do, this dataset is just what it was at the beginning: A collection of numbers with no meaning.

Preparing the MNIST Data

Now we reach the part where things finally start to *change*. Up until now, we've only **observed** the data. Now, we begin to **transform** it.

First, we reshape the images: A **28 × 28 matrix** → **flattened to arrays (or vectors)**

```
# Reshape image from (28, 28) → (784,)
train_images = train_images.reshape((60000, 28 * 28))
test_images = test_images.reshape((10000, 28 * 28))

print("Train Images Shape:", train_images.shape)
print("Test Images Shape:", test_images.shape)
```

Train Images Shape: (60000, 784)
Test Images Shape: (10000, 784)

Which means: Each image is now a vector of **784 numbers.**

What we did here is not "destroying the image." → We are rewriting it in a language the model understands. The spatial structure (rows and columns) is gone, but the information is still there.

💡 Think of it as for the training data: ***60000 samples and 784 features*** → corresponding to ***60000 labels***

Second, we normalize the data.

```
# Normalize data
train_images = train_images.astype("float32") / 255
test_images = test_images.astype("float32") / 255
```

Now comes something that often confuses people. The immediate question is: Why divide by **255**? Let's go back to what these numbers actually represent. Each pixel in the image has a value between: **0** → **black; 255** → **white.** So 255 is not random.

It is the **maximum possible intensity** in a grayscale image. So when we divide by 255, we are applying a simple idea: Scale everything into the range $[0, 1]$

Instead of: $0 \rightarrow 255$. We now have: $0.0 \rightarrow 1.0$

Why Does This Matter? Because neural networks are extremely sensitive to scale. If we leave values like: $0, 128, 255$, the model may learn slowly, become unstable, and struggle to converge

But when everything is in a small, consistent range, the learning process becomes smoother. Faster. More stable. More reliable. So normalization is not just a trick. It is a way to make learning *possible*.

💡 **A Very Important Rule (Do Not Ignore This)**
Let's talk about something that beginners often get wrong.
What about the labels? Should we normalize them too? Encode them differently? Scale them?

> **No.** And I want to make this very clear:
> **THE TARGET SHOULD NEVER BE RANDOMLY TRANSFORMED.**
> Because labels are not features. They are the **ground truth**. If your labels are: `[5, 0, 4, 1, 9, ...]`
> That already represents exactly what the model needs to learn.
> You only transform labels when the model explicitly requires it (e.g., one-hot encoding for classification) or the format is incompatible.
> But you do **not n**ormalize them, scale them, or randomly change their meaning.
> Because if you distort the labels, you are no longer teaching the model the correct answer.

At this point, we have transformed the data from raw matrices of pixel values into clean, normalized feature vectors So now the dataset looks like:

- `train_images` → shape `(60000, 784)` with values in `[0, 1]`
- `train_labels` → correct digit `(0–9)`

And only now, for the first time, the data is in a form where a neural network can actually begin to learn.

Next, we will build the model itself. You'll see how these 784 numbers turn into something that can recognize a handwritten digit.

Step 3. Build the Model

Now we arrive at the moment where everything starts to come alive → *Where Numbers Begin to Think*.

Up until now, we've only been preparing data. Cleaning it. Reshaping it. Translating it. But the data itself does nothing. What we need now is something that can **learn from it**. And this is where **neural networks** come in.

What Is a Neural Network (At This Stage)?

Forget the complex diagrams for a moment. At its core, a neural network is just a system of layers that transforms numbers into predictions.

Each layer contains:

- **Neurons** → small computation units
- **Weights** → how important each input is
- **Bias** → a small adjustment to shift the result

So when data flows through the network: ***Each neuron takes inputs → Multiplies them by weights → Adds a bias → And produces an output.***

But here's the problem. If we stop there, nothing interesting happens. Because without something extra, the entire network becomes just a series of linear transformations. And linear systems are very limited. They cannot capture complex patterns like curves, shapes, and especially mages.

This Is Why We Need Activation Functions

Activation functions are the **decision-makers** of the network. They answer the question: ***"Should this neuron activate... or not?"***

They take the raw output of a neuron and transform it into something meaningful. You can think of them as: ***The moment where calculation becomes decision.***

There are a few important ones you will see again and again:

- **ReLU (Rectified Linear Unit)** → most common for hidden layers
- **Sigmoid** → outputs values between 0 and 1 (probability-like)
- **Tanh** → outputs between -1 and 1
- **Softmax** → converts outputs into probabilities across multiple classes

Choosing Activation Functions (Not Randomly)

Now here's something important. We don't just pick activation functions randomly.

- **Hidden Layers:** For hidden layers, we usually use **ReLU** → Why? It is simple. It is fast. It works well in practice. So most of the time: $\text{Hidden layers} = \text{ReLU}$
- **Output Layer:** This is where things must be precise. Because the output layer defines what kind of problem we are solving.
  - In our case (MNIST): We have digits from **0 to 9.** That means **10 possible classes.**
  - So we need **10 neurons** → One for each class
  - And the activation function? **Softmax** → Because Softmax converts outputs into: A probability distribution
  - So the model can say something like: 0 → 0.01; 1 → 0.02; 2 → 0.05...; 7 → 0.85. Meaning: "I am 85% confident this is a 7."

The Model Structure

Now let's translate all of that into actual structure:

```
from tensorflow import keras
from tensorflow.keras import layers
```

```
# Build the model
hidden_layer1 = layers.Dense(512, activation="relu")
hidden_layer2 = layers.Dense(256, activation="relu")
output_layer = layers.Dense(10, activation="softmax")

model = keras.Sequential([hidden_layer1, hidden_layer2,
                          output_layer])
```

What Did We Just Build? Let's break it down clearly:

- First layer → 512 neurons (ReLU)
- Second layer → 256 neurons (ReLU)
- Output layer → 10 neurons (Softmax)

So the flow becomes: $784 \text{ inputs} \rightarrow 512 \rightarrow 256 \rightarrow 10 \text{ outputs}$

This is the structure of the network. This is the **architecture**.

A very natural question comes up:

> ***With this structure... how many weights does the model actually need to learn?***

Let's go layer by layer.

- **Input → Hidden Layer 1 (**784 input neurons → 512 neurons in the first hidden layer)

Each input neuron connects to **every neuron** in the next layer. But each neuron in the hidden layer also has a **bias**. So we add:

$$Weights = 784 \times 512 + Bias(512)$$

- **Hidden Layer 1 → Hidden Layer 2 (**$512 \rightarrow 256$)

$$Weights = 512 \times 256 + Bias(256)$$

- **Hidden Layer 2 → Output Layer (**$256 \rightarrow 10$)

$$Weights = 256 \times 10 + Bias(10)$$

Why Do We Always Include Bias? This is important. Bias is not just an extra number. It allows each neuron to **shift its activation independently.** Without bias, every neuron is forced to pass through the origin. Which means the model becomes less flexible.

→ So the rule is simple: ***Each neuron has exactly one bias***

When you add everything together, you realize something: Even a "simple" network is learning **hundreds of thousands of parameters.**

In other words, right now, the model knows nothing. The weights? We know how many it needs to learn, but the values themselves are just random. The predictions? Meaningless.

We have built the **brain.** But we haven't taught it anything yet.

The Missing Piece

At this stage, the model is just a structure. To make it learn, we still need to define:

- How it measures error (**loss function**)
- How it updates weights (**optimizer**)
- How learning actually happens (**backpropagation**)

In other words, we've built the system. But we haven't defined the rules of learning. And that... is exactly what happens in the next step:

Step 4. Compile the Model

We now step into the **real mechanics** of neural networks → *Where the Network Learns How to Learn*

Up until this point: **We loaded the data → We prepared it → We built the model**

But the model is still empty. It has layers, neurons, connections. But no understanding.

This Is Where Learning Begins

When we compile the model, we are not training it yet. We are doing something more subtle: We are defining **how the model will learn**.

```
# Compile the model
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

Simple code. But behind it, is the entire learning process of neural networks.

What Is Actually Happening Inside? Let's slow this down. Because this is the moment where everything connects.

The Flow of a Neural Network

Imagine one sample—just one image. It enters the network.

(1) Forward Pass

- The input (784 features) goes into the first layer
- Each neuron applies weights and bias
- The result passes through an activation function
- Then flows to the next layer
- And continues layer by layer

Until finally: The network produces an output → probabilities (via Softmax)

At this point, the model is making a guess. But we don't care about guesses. We care about: How wrong the guess is.

(2) Loss (Measuring Error)

This is where the loss function comes in. In our case: `sparse_categorical_crossentropy`

What does it do? It compares the predicted probabilities and the actual label. And produces a number: ***How bad was the prediction?***

- If the model is confident and correct → low loss
- If the model is wrong (especially confidently wrong) → high loss
- And this number drives everything that comes next.

(3) Backpropagation (The Core Mechanic)

Now we reach the heart of neural networks: Backpropagation. The idea is simple: ***We know the error (loss) → Now we ask: "Which weights caused this error?"***

So we move backward through the network: From **output layer → hidden layers → input connections.** And we compute something called ***Gradients*** (how much each weight contributed to the error). This is not guessing. It is calculated precisely using calculus.

(4) Optimizer (Fixing the Mistakes)

Now we have the gradients. But gradients alone do nothing. We need something to update the weights. That's the job of the optimizer. In our case: `rmsprop`

You can think of the optimizer as the one who adjusts the model after every mistake. It takes the gradients and applies Gradient Descent. Meaning:

- If a weight increases error → decrease it
- If a weight reduces error → reinforce it

So over time, the model slowly moves toward better predictions.

(5) Putting It All Together

The full loop looks like this:

$$\text{Input} \rightarrow \text{Prediction} \rightarrow \text{Loss} \rightarrow \text{Backpropagation} \rightarrow \text{Weight Update} \rightarrow \text{Repeat}$$

And this loop happens: Thousands of times → Across all samples → Over multiple passes

> 💡 **Let's Address the Confusion**
> A very important question comes up here: **Are neurons the samples… or the features?** They are **the features**.

Each input neuron corresponds to **one feature** of the data. In the MNIST case, that means: ***One neuron = one pixel***.
Since each image has 28 × 28 pixels: We have **784 input neurons**.
Now here's the interesting part. What does each neuron actually "hold"? Not just one value. Across the entire dataset, each neuron represents: A **vector of values across all samples**.
So for a single neuron (say pixel at position [0,0]): It takes one value from each image. Across 60,000 training images
Which means: That neuron corresponds to a vector of size **60,000**
So we can see the data in two ways:

- **Sample view** → 60,000 images, each with 784 features
- **Feature view** → 784 neurons, each with 60,000 values

Both are correct. But in neural networks, we think in terms of **features flowing through neurons.** Not samples being stored inside them.

At this stage, everything is finally connected:

- Weights → what the model learns
- Forward pass → making predictions
- Loss → measuring mistakes
- Backpropagation → tracing errors
- Optimizer → correcting the model

And this is the most important idea: A neural network is not just a structure. It is a system that continuously corrects itself.

Right now, the model knows how to learn. It knows how to measure error. It knows how to adjust itself. But it still hasn't learned anything yet. Because learning only begins when we do one thing: Start training. And that... is exactly what happens in the next step.

Step 5. Train and Validate the Model

Everything is finally in place. The data is prepared. The model is built. The learning rules are defined. Now, we do the one thing that turns all of this into intelligence: **We start training.**

```
# Fit the model
model.fit(train_images, train_labels, epochs=20, batch_size=128, valida
tion_split=0.2)
```

Simple. But this line triggers the entire learning process we described earlier.

When we call `model.fit(...)`, the model begins this loop:

$$\text{Input} \rightarrow \text{Prediction} \rightarrow \text{Loss} \rightarrow \text{Backpropagation} \rightarrow \text{Update} \rightarrow \text{Repeat}$$

And it does this over and over. Across all samples. Until patterns start to emerge.

But now we need to understand two very important concepts: **Epoch** and **Batch Size**

Epoch — One Full Pass Through the Data

An **epoch** means the model has seen **every training sample once**

In our case: 60,000 training images → 1 epoch = the model processes all 60,000 images

So when we write: `epochs=20`. It means: The model will go through the entire dataset **20 times.** Why repeat? Because learning is not instant. Each pass: Refines the weights; Reduces the error; Improves the predictions.

Batch Size — How Much the Model Sees at Once

Now here's the important detail: The model does NOT process all 60,000 images at once. Instead, it splits the data into smaller groups: **Batches**

In our case: `batch_size=128`. This means: The model looks at **128 images at a time** → Computes predictions → Calculates loss → Updates weights. Then moves to the next 128... So within **one epoch**, the model performs many updates.

So in short:

- **Epoch** → how many times we go through the dataset
- **Batch size** → how much we process before updating

Training is not: ***"See everything → update once"***. It is: ***"See a little → update → repeat → refine"***

This design makes learning more stable, more efficient, less memory-intensive

- Small batches: Faster updates. Noisy but flexible learning
- Large batches: More stable. Slower to adapt

Validation Data

Finally, we introduce **validation data**. Here, **20% of the training data is set aside for validation**.

These validation samples are **not used for training**. The model does **not learn from them**, does **not update weights using them**, and does **not "see" them during learning**.

Instead, they are used to evaluate the model **during training.** So after each epoch:

- The model trains on 80% of the data
- Then it is tested on the remaining 20%

This allows us to answer a critical question: ***Is the model actually learning patterns... or just memorizing?*** Because if the model performs well on training data but poorly on validation data: It is memorizing. But if it performs well on both, it is learning.

At this stage, the model is no longer random. Weights are being adjusted. Patterns are being discovered

And this is the moment where something subtle happens. The model begins to **understand the data**. Not like a human. But in its own way. Through numbers, weights, and repeated correction.

Next, we will **observe this learning process,** by looking at loss and accuracy as they evolve over time.

Step 6. Fit the Model and Monitor Performance

Now... training has started. But training alone is not enough. We need to **observe what is happening**.

When the model trains, it doesn't just learn silently. It records everything.

```
# Record the model in history
history = model.history

# Print the metrics
print("Train Loss:", history.history["loss"][-1])
print("Validation Loss:", history.history["val_loss"][-1])
print("Train Accuracy:", history.history["accuracy"][-1])
print("Validation Accuracy:", history.history["val_accuracy"][-1])
```

```
Train Loss: 1.5312090908992104e-05
Validation Loss: 0.10919255763292313
Train Accuracy: 1.0
Validation Accuracy: 0.9828333258628845
```

What Is `history`? It is a record of the entire training process. At every epoch, it stores: Loss; Accuracy; Validation loss; Validation accuracy

And What Does `[-1]` Mean? It simply means take the **last value.** Which corresponds to the **final result after training**

So instead of looking at all epochs, we are extracting:

- Final training performance
- Final validation performance

Why This Matters - Because these numbers tell us:

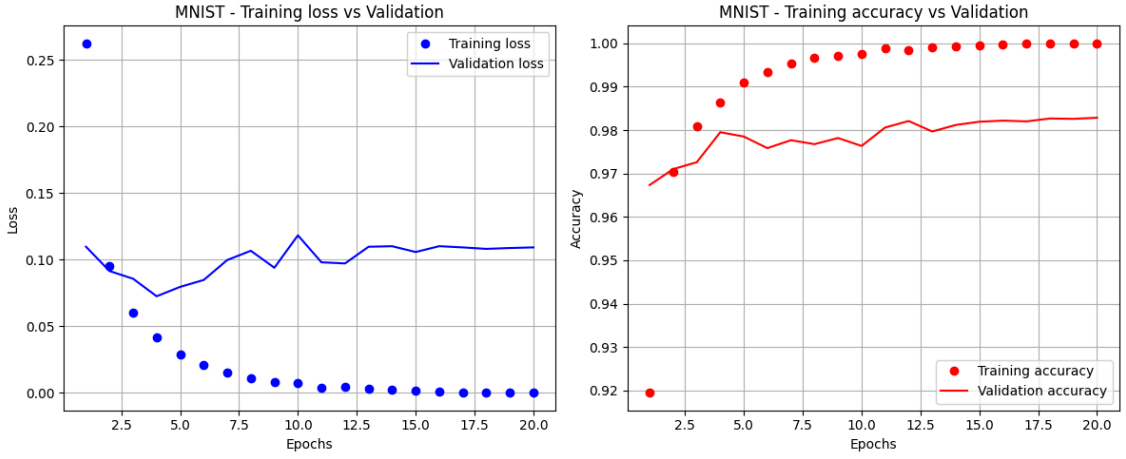
- How well the model learned
- Whether it generalized
- Whether it overfitted

And more importantly, this `history` object is not just for printing. It is the foundation for **visualizing learning**. And that's where the real insight begins.

Step 7. Select the Best Model (Epoch Selection)

We don’t just train a model and accept the final result. We **observe its behavior over time**.

```
# Plot the Train Loss, Validation Loss, Train Accuracy, Validation Accu
racy
import matplotlib.pyplot as plt


loss = history.history['loss']
val_loss = history.history['val_loss']
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
epochs = range(1, len(loss) + 1)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(epochs, loss, 'bo', label='Training loss')
ax[0].plot(epochs, val_loss, 'b', label='Validation loss')
ax[0].set_title('MNIST - Training loss vs Validation')
ax[0].set_xlabel('Epochs')
ax[0].set_ylabel('Loss')
ax[0].legend()
ax[0].grid(True)

ax[1].plot(epochs, acc, 'ro', label='Training accuracy')
ax[1].plot(epochs, val_acc, 'r', label='Validation accuracy')
ax[1].set_title('MNIST - Training accuracy vs Validation')
ax[1].set_xlabel('Epochs')
ax[1].set_ylabel('Accuracy')
ax[1].legend()
ax[1].grid(True)

plt.tight_layout()
plt.show()
```



These curves tell a story: Training improves steadily. Validation improves, then may stop or degrade

So the question becomes: **When was the model actually at its best?**

A simple and powerful rule:

- Choose the epoch where **validation loss is lowest**
- And **validation accuracy is highest**

Because that is the point where the model learns well without overfitting.

Step 8. Evaluate on the Test Set

This is the model's final exam. Now we test the model on **completely unseen data**.

```
import numpy as np

# Evaluate on test set
y_pred = model.predict(test_images)
y_pred = np.argmax(y_pred, axis=1)
print("Test Accuracy:", np.mean(y_pred == test_labels))
print("Test Loss:", model.evaluate(test_images, test_labels)[0])
```

```
Test Accuracy: 0.9856
Test Loss: 0.08994387090206146
```

Why `argmax`? Because the model outputs **probabilities**: $[0.01, 0.02, 0.05, \ldots, 0.85]$. We need to convert this into a final decision. So `argmax` simply means: Pick the index with the highest probability. Which becomes the predicted digit.

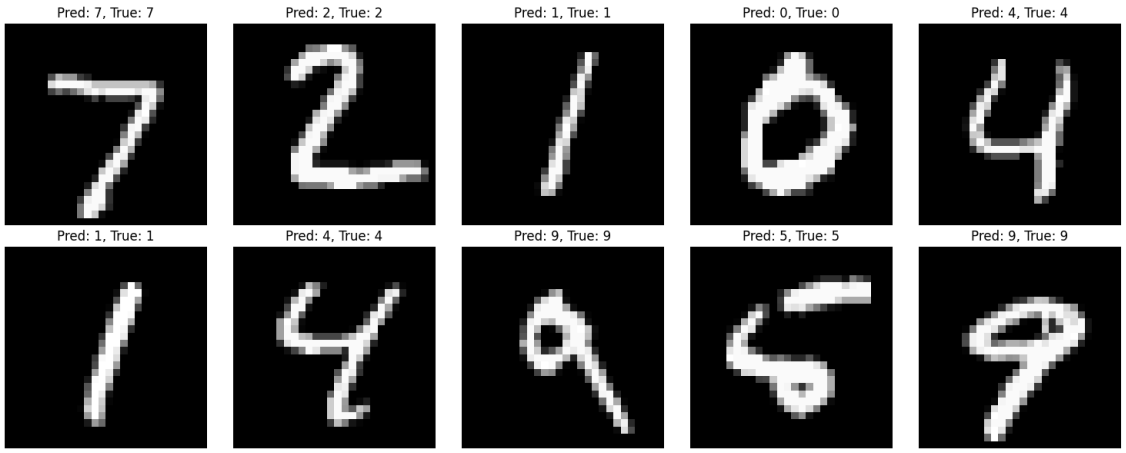
This step tells us one thing: ***Can the model handle data it has never seen before?***

Step 9. Prediction

Finally, we use the model like a real system.

```
# Predict on new images
import numpy as np
import matplotlib.pyplot as plt

# Get the first 10 images from the test set
new_images = test_images[:10]
new_labels = test_labels[:10]

# Create a figure and a 2x5 subplot grid
fig, axes = plt.subplots(2, 5, figsize=(15, 6))
axes = axes.flatten() # Flatten the 2x5 array of axes for easy iteratio
n

# Predict for each image and display the result
for i in range(10):
    current_image = new_images[i]
    true_label = new_labels[i]
    # Reshape to match the model's input shape (1, 28 * 28)
    image_input = current_image.reshape(1, 28 * 28)
    prediction = model.predict(image_input, verbose=0)
    predicted_label = prediction[0].argmax()

    print(f"Prediction: {predicted_label}, True Label: {true_label}")

    # Display the image in the subplot
    axes[i].imshow(current_image.reshape(28, 28), cmap='gray')
    axes[i].set_title(f"Pred: {predicted_label}, True: {true_label}")
    axes[i].axis('off')

plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
plt.show()
```

```
Prediction: 7, True Label: 7
Prediction: 2, True Label: 2
Prediction: 1, True Label: 1
Prediction: 0, True Label: 0
Prediction: 4, True Label: 4
Prediction: 1, True Label: 1
Prediction: 4, True Label: 4
Prediction: 9, True Label: 9
Prediction: 5, True Label: 5
Prediction: 9, True Label: 9
```



Retraining the model

Optionally, you can retrain the model with the optimal epoch:

```
# Retrain the model with the optimal epoch
model = keras.Sequential([hidden_layer1, hidden_layer2,
                          output_layer])
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
# Based on the plot, we choose epoch=4
model.fit(train_images, train_labels, epochs=4, batch_size=128, validat
ion_split=0.2)
# Compute loss
y_pred = model.predict(test_images)
y_pred = np.argmax(y_pred, axis=1)
print("Test Accuracy:", np.mean(y_pred == test_labels))
print("Test Loss:", model.evaluate(test_images, test_labels)[0])
```

Test Accuracy: 0.9848
Test Loss: 0.09081248193979263

You can download the full code here:

neural_network.py

Summary

What we've just gone through is not just an example. It is the **full lifecycle of a neural network**.

We started with nothing. Just raw data—unstructured, unreadable, meaningless to a machine. Then step by step, we transformed it:

- We **loaded the data**
- We **prepared it** so the model could understand
- We **built the architecture**—layers, neurons, activation functions

At that point, we had a structure, but no intelligence. So we defined how it learns:

- **Loss** to measure error
- **Optimizer** to correct it
- **Backpropagation** to connect everything

And then, we let the model learn. Through training:

- It saw the data again and again (**epochs**)
- It learned in small steps (**batches**)
- It improved gradually

But we didn't trust it blindly. We monitored:

- Training performance
- Validation performance

To answer one critical question: "***Is it learning... or just memorizing?"***

Then we went further. We **visualized the learning process**, selected the **best epoch**, and even retrained the model more efficiently.

Finally, we tested it on unseen data (**evaluation**), on real inputs (**prediction**). And that's where everything becomes real. Because at the end of this pipeline, the model is no longer just code. It becomes a system that can take raw input, and make decisions.

So if you take one idea from this entire section, let it be this: Deep Learning is not about layers or equations. It is about a **process**—a structured way of turning data into understanding.

And once you understand this workflow, you can apply it to anything: Images; Text; Audio; Predictions; Decisions

Different problems. Same pipeline. And that is the foundation of Neural Networks.

References:

https://www.tensorflow.org/datasets/keras_example

https://www.kaggle.com/code/prashant111/mnist-deep-neural-network-with-keras

Next Section:

5. Neural Networks: Binary Classification